

APPLICATION OVERVIEW AND TECHNOLOGY STACK

Kith.

The private-by-default messenger. End-to-end encrypted before your message ever leaves the phone.

A production-grade, open-source mobile messenger built on Expo SDK 57 with a zero-knowledge Hono relay. Real X3DH-lite key agreement, XChaCha20-Poly1305 sealed messages, passwordless key-based login, and a seed-phrase recovery only the user holds.

End-to-end encrypted

Zero-knowledge relay

Passwordless identity

BIP39 recovery

Encrypted local history

No phone number

Open source (MIT)

PLATFORM

iOS and Android

FRAMEWORK

Expo SDK 57

VERSION

1.0.0

LANGUAGE

TypeScript (strict)

AUTHOR

Aashir Athar

github.com/aashir-athar/kith · linkedin.com/in/aashirathar · x.com/aashirathar

1. What Kith is

Kith is a privacy-first mobile messenger. It is built so users do not have to trust the server. Every message is sealed on the device with a key the relay never learns, the account is a cryptographic keypair rather than a phone number, and the only account backup is a twelve-word phrase the user writes down. If the relay operator, a network attacker, or a legal request reaches the database, what they get is ciphertext and routing metadata, never conversations.

The repository is a full stack: the Expo app, a shared cryptography package that runs byte-for-byte the same on the client and in the server test suite, and a Hono relay that can be brought up locally with one Docker command.

Design principle: ship behavior, not theatre. The app only exposes what is actually wired to the encrypted transport. Features that are not implemented yet are honestly absent in a live build rather than faked. Destructive controls do real work, security claims derive from real state, and no screen overclaims what the cryptography provides.

2. Feature set

Shipped and wired to the real encrypted transport

- **Live** One-to-one text messaging, end-to-end encrypted
- **Live** Send, edit, and delete-for-everyone over the relay
- **Live** Delivery and read receipts
- **Live** Typing indicators and presence
- **Live** History hydration and gap-detectable sync
- **Live** Encrypted-at-rest local message store
- **Live** Passwordless registration and login
- **Live** Cross-device restore from a recovery phrase
- **Live** Real account deletion (relay plus device wipe)
- **Live** Local block, mute, archive, and QR code add

Designed and previewed in the offline demo, not yet on the relay

- **Planned** Media, voice notes, and stickers
- **Planned** Voice and video calls
- **Planned** Communities and channels
- **Planned** Double Ratchet forward secrecy
- **Planned** Multi-device sync for one identity
- **Planned** Server-side moderation review

In a live build, the demo-only surfaces are hidden and unreachable rather than shown as working controls.

3. Security and cryptography

Kith uses an X3DH-lite handshake. A sender fetches the recipient's signed prekey bundle, performs a set of X25519 Diffie-Hellman operations, derives a session key with HKDF-SHA256, and seals the message with XChaCha20-Poly1305. The routing header is bound as authenticated data, so a relay cannot swap the key path without breaking decryption.

Cryptographic primitives

Identity signatures	Ed25519
Auth challenge	Ed25519 signature
Key agreement	X25519 ECDH (X3DH-lite)
Message encryption	XChaCha20-Poly1305 AEAD
Key derivation	HKDF-SHA256
Local store at rest	XChaCha20-Poly1305 + device key
Recovery phrase	BIP39 to seed to identity
Wire encoding	Hex envelopes

Threat model

The relay can see: who talks to whom, timing, message sizes, and public keys.

The relay cannot see: message contents, or any secret key. Secret keys never leave the device secure enclave.

Honest limit (v1): the session key is static per handshake, so Kith provides confidentiality, sender authentication, and asynchronous first contact, but not yet forward secrecy or post-compromise security. A Double Ratchet is the upgrade path, and the app never claims perfect forward secrecy.

Identity, login, and recovery

Randomness is injected, never assumed: the client uses `expo-crypto`, the server and tests use the Node CSPRNG, so there is exactly one audited crypto implementation and no `get-random-values` polyfill dependency. Login is passwordless: the client proves control of its Ed25519 identity by signing a server challenge. The account itself is derived from a twelve-word BIP39 phrase, so the same phrase reproduces the same identity on any device with no server-side secret. On restore, the device re-derives the identity, logs in by key, and rotates fresh prekeys so peers can seal to it again.

Data protection posture

- Secret keys, the local data key, the session token, and the recovery phrase live only in `expo-secure-store` (iOS Keychain / Android Keystore).
- The local message history is persisted as ciphertext in AsyncStorage, sealed with a 32-byte data key held in the secure enclave. Plaintext never touches disk.
- The relay stores only opaque ciphertext, routing metadata, and public key material. Account deletion removes the user's data from the relay and wipes every key on the device.

4. Architecture

A small monorepo with one rule: the cryptography is written once and shared, so the code exercised by the server test suite is the same code that runs on the phone.

kith (Expo app)

- Expo Router screens with real native tabs
- Zustand stores (session, chat with encrypted persist)
- On-device crypto: keystore, e2e, mnemonic
- REST client and reconnecting WebSocket

@kith/shared (isomorphic TS)

- X3DH-lite: seal, open, identityFromSeed
- Zod DTOs and wire types shared by both sides
- Self-contained deps so it resolves everywhere

kith/server (Hono relay)

- REST: auth, keys, conversations, account
- WebSocket gateway with per-user fan-out
- Passwordless key-challenge auth, single-use tickets

Data layer

- Postgres: ciphertext and public key material only
- Redis: single-use realtime tickets and pub/sub fan-out
- Drizzle ORM with typed schema and migrations

Repository layout

```
kith/  
  src/app/           Expo Router routes (src/app structure, native tabs)  
  src/crypto/        keystore (secure-store), e2e, mnemonic, random  
  src/stores/        Zustand: session + chat (encrypted persist)  
  src/api/           REST client, reconnecting socket, query client  
  src/net/           transport config, messaging singleton, secure storage  
  shared/            @kith/shared: X3DH-lite crypto + Zod DTOs  
  server/            Hono relay: routes, ws gateway, Drizzle schema, migrations  
  server/docker-compose.yml  Postgres 16 + Redis 7 + relay
```

Data model (relay)

users, devices, signed_prekeys, one_time_prekeys, conversations, conversation_participants, messages. Receipts use per-participant cursors (last_delivered_seq / last_read_seq); each conversation carries a monotonic next_seq for gap-detectable ordering. Message ciphertext is stored as **bytea**, routing metadata as **jsonb**.

5. Technology stack

Complete dependency inventory with exact pinned or resolved versions, grouped by role. Every package is installed through the Expo tooling to stay SDK-57 compatible.

Client core

PACKAGE	VERSION	ROLE
expo	~57.0.2	App framework and native runtime (New Architecture)
react-native	0.86.0	Native UI runtime
react / react-dom	19.2.3	UI library (React Compiler enabled)
expo-router	~57.0.3	File-based routing and native tabs
react-native-screens	4.25.2	Native screen and tab primitives
react-native-safe-area-context	~5.7.0	Safe-area insets
react-native-gesture-handler	~2.32.0	Native gestures
react-native-reanimated	4.5.0	Animations (worklets)
react-native-worklets	0.10.0	Worklet runtime for Reanimated
react-native-web	~0.21.0	Web target support

UI, media, and interaction

PACKAGE	VERSION	ROLE
@shopify/flash-list	2.0.2	High-performance message and chat lists
@expo/ui	~57.0.3	Native UI building blocks
expo-glass-effect	~57.0.0	Liquid Glass surfaces on iOS 26
expo-blur	~57.0.0	Blur surfaces (older iOS fallback)
expo-image	~57.0.0	Image rendering and caching
expo-linear-gradient	~57.0.0	Gradients (avatars, accents)
expo-symbols	~57.0.0	SF Symbols
lucide-react-native	^1.23.0	Icon set
react-native-svg	15.15.4	Vector rendering
react-native-qrcode-svg	^6.3.21	QR code for in-person add
expo-camera	~57.0.0	QR scanning (on-device)
expo-haptics	~57.0.0	Tactile feedback
expo-clipboard	~57.0.0	Copy message text

State, data, and forms

PACKAGE	VERSION	ROLE
zustand	^5.0.14	App state (session, chat with encrypted persist)
@tanstack/react-query	^5.101.2	Server-state caching
@react-native-async-storage/async-storage	2.2.0	Ciphertext-only local storage
react-hook-form	^7.81.0	Form handling
zod	^4.4.3	Runtime validation and shared contracts
ulid	^3.0.2	Sortable client ids

Cryptography and secure storage

PACKAGE	VERSION	ROLE
@noble/curves	^2.2.0	Ed25519 signatures and X25519 key agreement
@noble/ciphers	^2.2.0	XChaCha20-Poly1305 AEAD
@noble/hashes	^2.2.0	HKDF-SHA256 and hashing utilities
@scure/bip39	^2.2.0	Recovery-phrase to seed
expo-secure-store	~57.0.0	Keychain / Keystore for all secrets
expo-crypto	~57.0.0	Injected client CSPRNG

Device and platform

PACKAGE	VERSION	ROLE
expo-splash-screen	~57.0.2	Splash handoff (no light-mode flash)
expo-status-bar / system-ui	~57.0.0	Status bar and system UI theming
expo-font	~57.0.0	Font loading
expo-constants / device / network	~57.0.x	Runtime, device, and connectivity info
expo-linking / web-browser	~57.0.x	Deep links and external links

Backend relay

PACKAGE	VERSION	ROLE
hono	4.12.28	HTTP framework (REST routes)
@hono/node-server	2.0.8	Node server with WebSocket upgrade
ws	8.21.0	WebSocket server
postgres	3.4.9	Postgres driver (postgres.js)
drizzle-orm	0.45.2	Typed schema, queries, migrations
ioredis	5.11.1	Redis client (tickets, pub/sub fan-out)
zod	4.4.3	Request validation (shared contracts)
dotenv	^17.0.0	Environment configuration
@kith/shared	file: ../shared	Shared crypto and DTOs

Shared package (@kith/shared)

PACKAGE	VERSION	ROLE
@noble/curves	2.2.0	Isomorphic key agreement and signatures
@noble/ciphers	^2.0.0	Isomorphic AEAD
@noble/hashes	^2.0.0	Isomorphic HKDF and hashing

zod

^4.4.0

Shared DTO schemas

Language, tooling, and testing

TOOL	VERSION	ROLE
typescript	~6.0.3	Strict typing across app, shared, and server
tsx	^4.19.0	Server run and test runner loader
node:test	Node 20+	Test runner (no framework)
@electric-sql/pglite	^0.5.4	In-process Postgres for real integration tests
drizzle-kit	0.31.10	Migration generation
Docker Compose	Postgres 16 + Redis 7	Local backend orchestration
EAS Build	Expo	Device and store builds

6. Engineering quality

Type safety

Strict TypeScript everywhere, including `noUncheckedIndexedAccess` and `noImplicitOverride`. App, shared, and server all typecheck clean.

Tested where it matters

16 tests: crypto vectors plus PGLite integration for conversations, messages, receipts, edit/delete, recovery determinism, and account erasure. All green.

No leaks

Timers, sockets, subscriptions, and effects are released on close. The reconnecting socket cannot outlive `close()`. Server maps are bounded and the heartbeat clears on shutdown.

Honest UX

No dead controls, no faked delivery, and no security claims that outrun the cryptography. Unwired features are hidden in a live build, not simulated.

7. Configuration and deployment

A single client variable, `EXPO_PUBLIC_API_URL`, points the app at a relay and switches it from the offline demo to the real encrypted backend. The relay reads `DATABASE_URL`, `REDIS_URL`, `SESSION_SECRET`, and `PORT`. The backend runs locally with `docker compose up` (Postgres 16, Redis 7, relay) and deploys as a standard Node service behind TLS, with the socket upgrading to WSS. Full instructions are in `zero-to-deploy.md`.